

Архитектуры нейросетей

CNN, RNN, LSTM

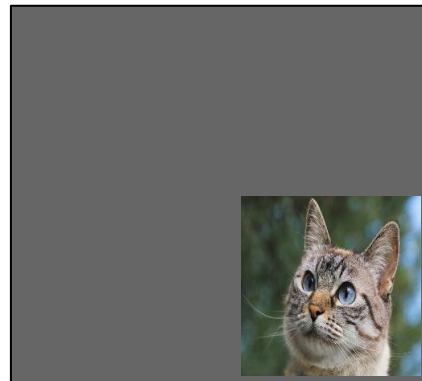
Напоминание: некоторые типы нейросетей

- Multilayer Perceptron/Fully Connected Neural Network (многослойный перцептрон/полносвязная нейросеть) - нейросеть, в которой есть только линейные слои и функции активации. Может использоваться для классификации или регрессии;
- Convolutional Neural Network (CNN, сверточная нейросеть) - нейросеть со сверточными слоями. Архитектура используется в основном для обработки изображений;
- Рекуррентные нейросети (RNN, LSTM) - архитектуры, позволяющие обрабатывать цепочки последовательностей. Используются для обработки текстов;

Сверточные нейросети/Convolutional Neural Networks

Сверточные нейросети: интуиция

Преимущество: в отличие от полносвязных нейросетей, мы учитываем структуру данных. При помощи сверточных слоев мы можем обращать внимание на небольшие фрагменты изображения. Например, мы можем распознать, что, вне зависимости от примененных к изображению ниже преобразований, на нем изображен кот.



Сверточные нейросети: интуиция

Полносвязной нейросети обучение на рисунке 1 не помогло бы распознать кота на рисунке 2. Сверточная нейросеть же обращает внимание на малые фрагменты изображения, что позволяет ей делать более успешную классификацию.



Рис. 1

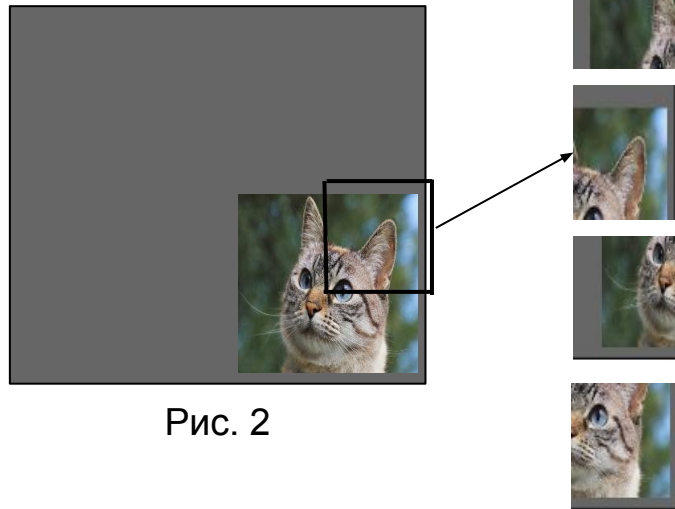


Рис. 2

Сверточные нейросети: принцип работы

Применение свёртки (kernel на рисунке справа): часть матрицы поэлементно умножается на свертку, затем значения суммируются, и окно свертки сдвигается дальше.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y
1																									
2		<u>Input</u>								<u>Kernel</u>						<u>Intermediate Output</u>									
3																									
4		1	0	1	0	2																			
5		1	1	3	2	1				0	1	0					7	5	3						
6		1	1	0	1	1				0	0	2					4	7	5						
7		2	3	2	1	3				0	1	0					7	2	8						
8		0	2	0	1	0																			
9																									
10		1	0	0	1	0																			
11		2	0	1	2	0				2	1	0					5	3	10				19	13	15
12		3	1	1	3	0				0	0	0					13	1	13				28	16	20
13		0	3	0	3	2				0	3	0					7	12	11				23	18	25
14		1	0	3	2	1																			
15																									
16		2	0	1	2	1																			
17		3	3	1	3	2				1	0	0					7	5	2						
18		2	1	1	1	0				1	0	0					11	8	2						
19		3	1	3	2	0				0	0	2					9	4	6						
20		1	1	2	1	1																			
21																									

Сверточные нейросети: принцип работы

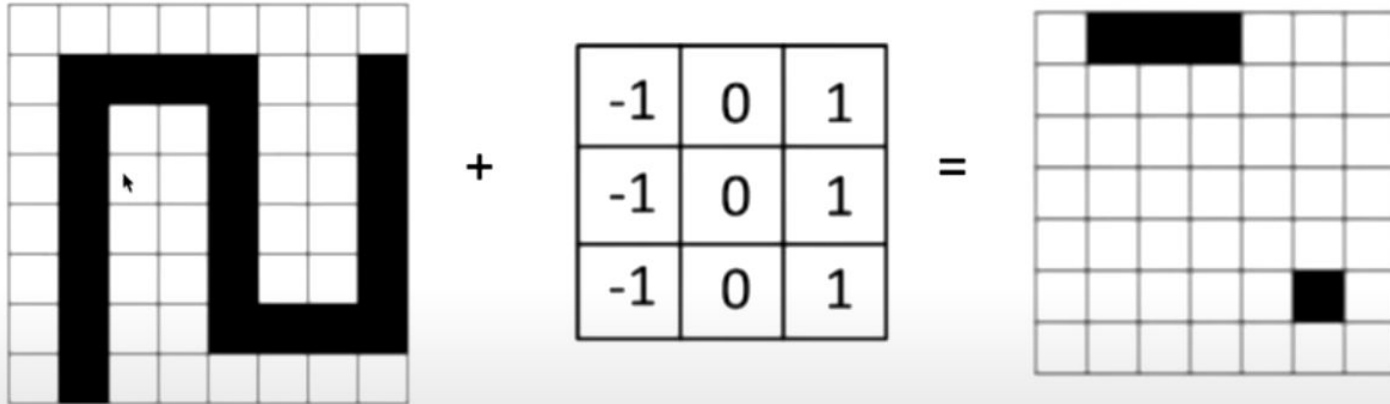
Input		Kernel	Intermediate Output
1010 1132 1101 2321 0201		010 002 010	753 475 728

```
In [1]: import numpy as np  
  
In [2]: matrix_part = np.array([[1,0,1], [1,1,3], [1,1,0]])  
  
In [3]: kernel = np.array([[0,1,0], [0,0,2], [0,1,0]])  
  
In [4]: np.sum(matrix_part * kernel)  
Out[4]: 7
```

```
In [5]: matrix_part = np.array([[0,1,0], [1,3,2], [1,0,1]])  
  
In [6]: kernel = np.array([[0,1,0], [0,0,2], [0,1,0]])  
  
In [7]: np.sum(matrix_part * kernel)  
Out[7]: 5
```

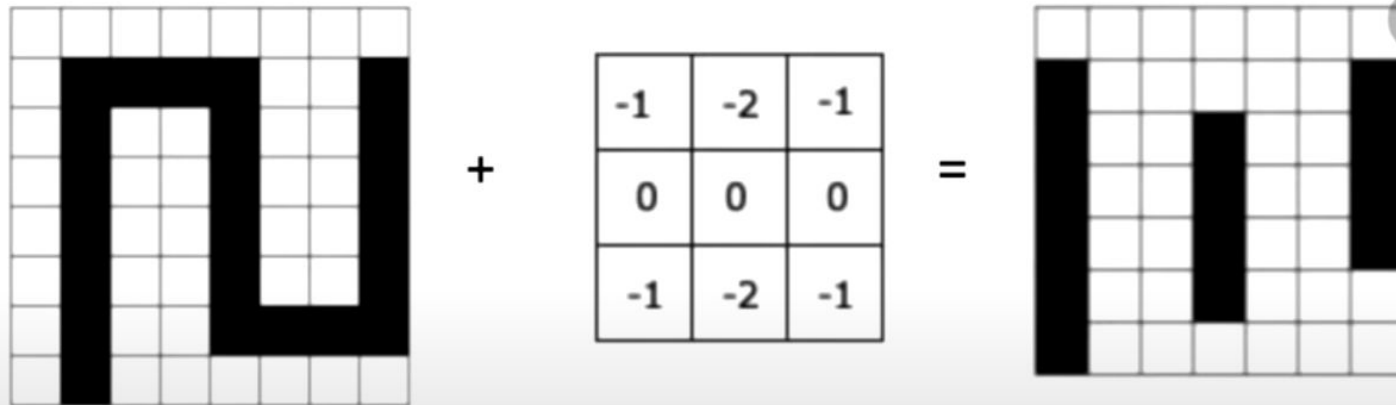
Что делают фильтры?

Как мог бы выглядеть фильтр, реагирующий на горизонтальные линии

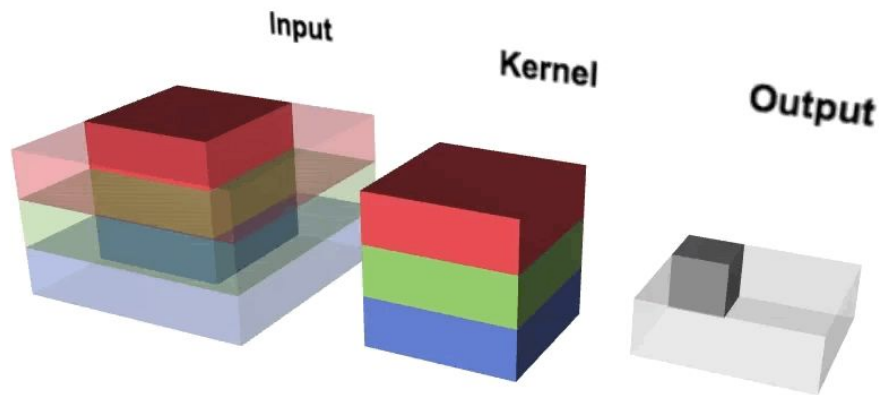


Что делают фильтры?

Как мог бы выглядеть фильтр, реагирующий на вертикальные линии



Сверточные нейросети: принцип работы



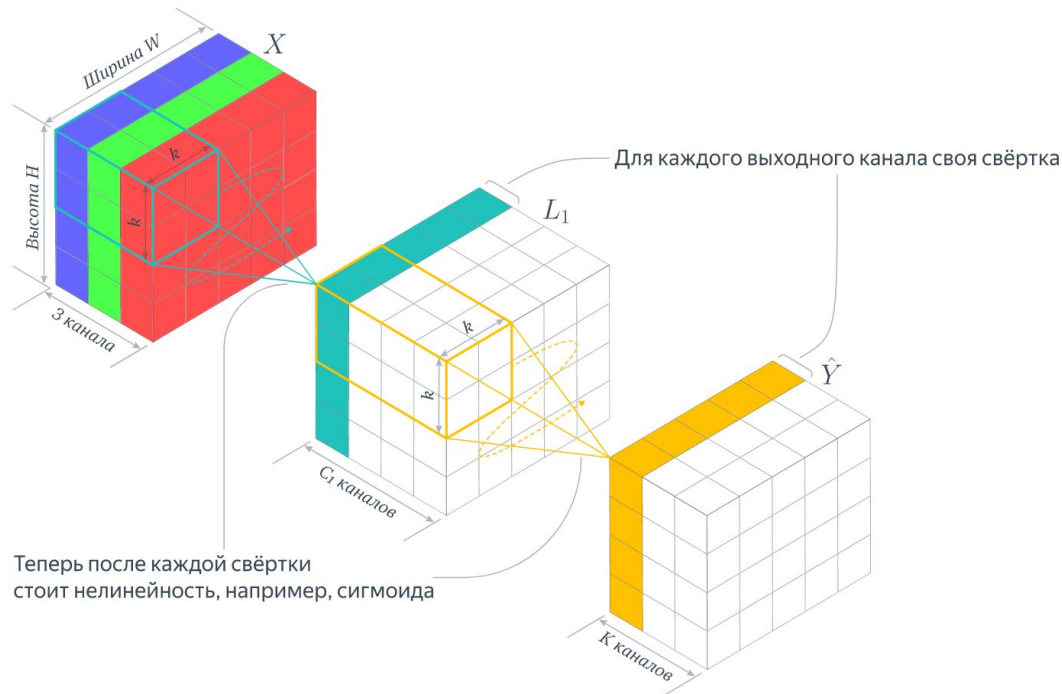
Сверточные нейросети: принцип работы

Поскольку в RGB-изображениях свертка применяется для трех цветовых каналов отдельно, в конце выходные матрицы суммируются поэлементно (output на рисунке справа).

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y
1																									
2		<u>Input</u>								<u>Kernel</u>						<u>Intermediate Output</u>									
3																									
4		1	0	1	0	2																			
5		1	1	3	2	1				0	1	0					7	5	3						
6		1	1	0	1	1				0	0	2					4	7	5						
7		2	3	2	1	3				0	1	0					7	2	8						
8		0	2	0	1	0																			
9																									
10		1	0	0	1	0																			
11		2	0	1	2	0				2	1	0					5	3	10				19	13	15
12		3	1	1	3	0				0	0	0					13	1	13				28	16	20
13		0	3	0	3	2				0	3	0					7	12	11				23	18	25
14		1	0	3	2	1																			
15																									
16		2	0	1	2	1																			
17		3	3	1	3	2				1	0	0					7	5	2						
18		2	1	1	1	0				1	0	0					11	8	2						
19		3	1	3	2	0				0	0	2					9	4	6						
20		1	1	2	1	1																			
21																									

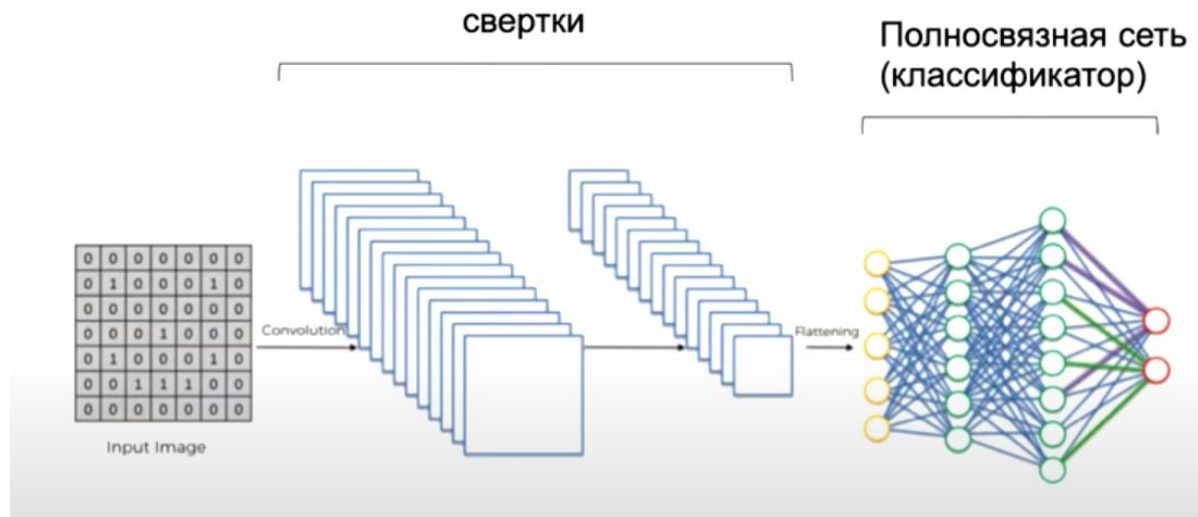
Сверточные нейросети: принцип работы

К первому слою применяется 4 разных свёртки/фильтра, которые образуют 4 новых канала, к каждому из которых применяется нелинейность. В нейросети может быть несколько сверточных слоев.



Сверточные нейросети: принцип работы

После последнего сверточного слоя матрица разворачивается в плоский вектор, и при помощи классификатора собираются финальные предсказания.



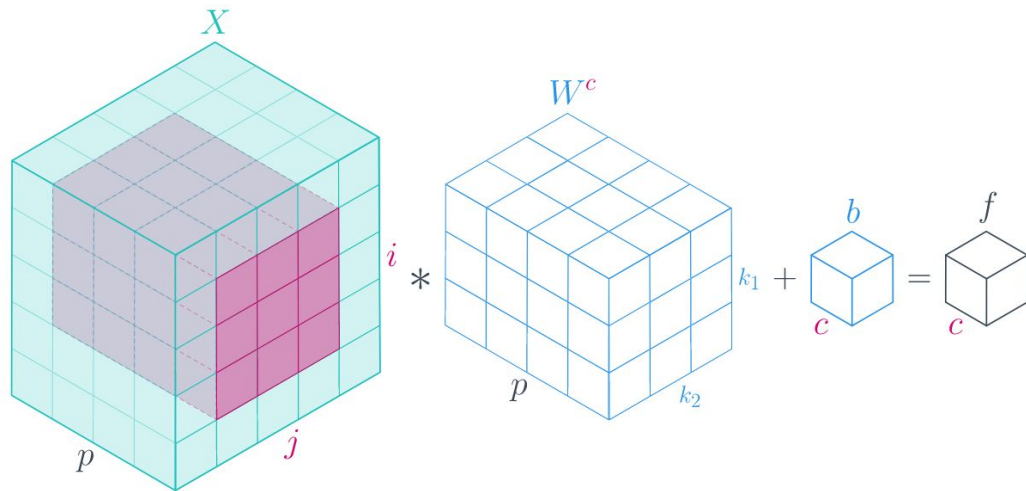
Сверточные нейросети: принцип работы

Вопрос: что является весами сверточной сети? Когда добавляется bias?

Сверточные нейросети: принцип работы

Ответ: веса CNN - это числа, составляющие свёртку. Bias добавляется перед вычислением скалярного произведения свертки и области матрицы, на которую она смотрит.

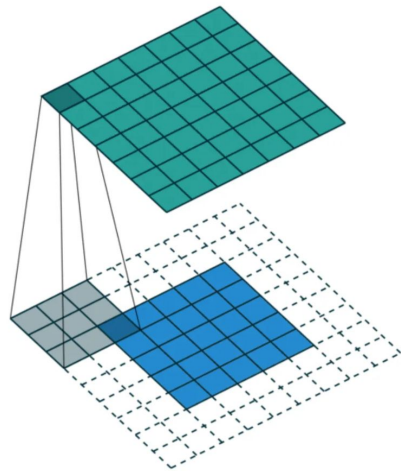
k — размер ядра; X — карта признаков $H \times W \times C_{in}$; f — карта признаков $H \times W \times C_{out}$



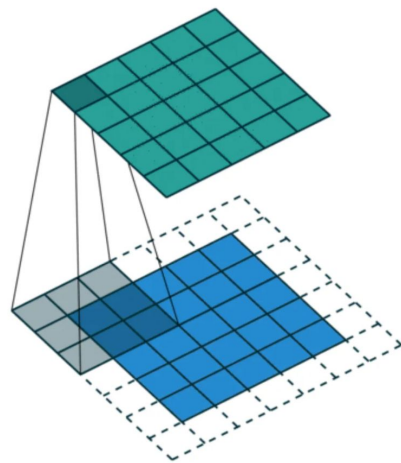
Padding

Если мы применим свертку шириной 3 к картинке шириной 5, мы получим output шириной 3. Что, если мы не хотим терять в размерности или вообще хотим ее увеличить?

Ответ: мы применяем padding, добавляя нули по краям изображения.



Full padding. Introduces zeros such that all pixels are visited the same amount of times by the filter. Increases size of output.



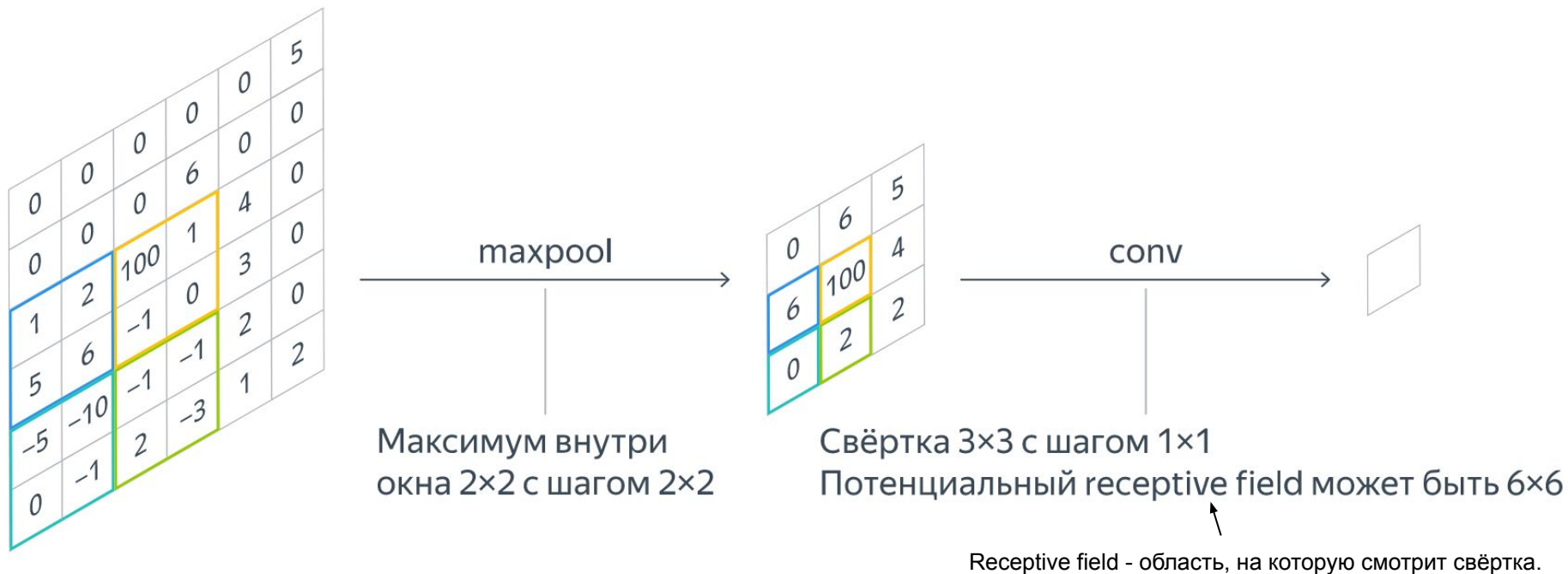
Same padding. Ensures that the output has the same size as the input.

Параметры сверточной нейросети

- Размерность входного слоя: (цветовые каналы*высота картинок*ширина картинок)
- Размерность свёртки: задаёт пользователь
- Количество свёрток, которое будет применено: задаёт пользователь
- Padding
- Stride/шаг: на сколько клеток в сторону двигается свертка.

Параметры сверточной нейросети: Pooling

Pooling - это способ уменьшить размерность полученного представления.



Можно ли применять свёртки для текстов?

Это делается нечасто, но это возможно, если предположить, что нам нужно найти в тексте феномен, который может встретиться где угодно, но будет указывать на отнесенность текста к какому-то классу. Например, в задаче анализа тональности проявления негативной или позитивной тональности могут встретиться в любом месте текста, так же, как кошка может встретиться на любом месте на картинке.

Рекуррентные нейросети/Recurrent Neural Networks

В чем минусы применения сверточных нейросетей для текстов?

В чем минусы применения сверточных нейросетей для текстов?

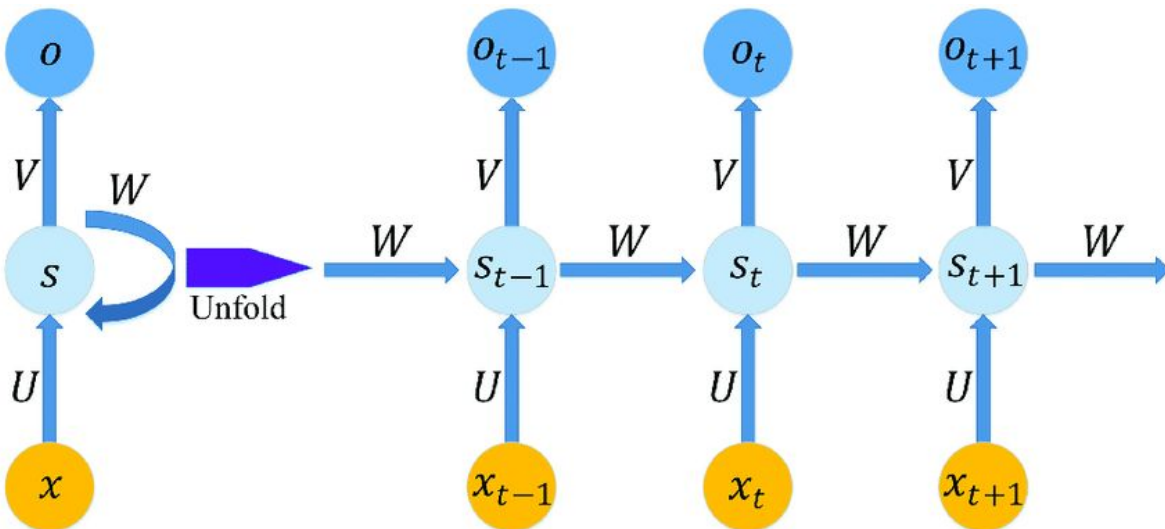
В тексте информация изложена последовательно, поэтому нам бы хотелось рассматривать слово в контексте.

Значит, нам нужна архитектура, которая способна запоминать предыдущие слова.

Рекуррентные нейросети: архитектура

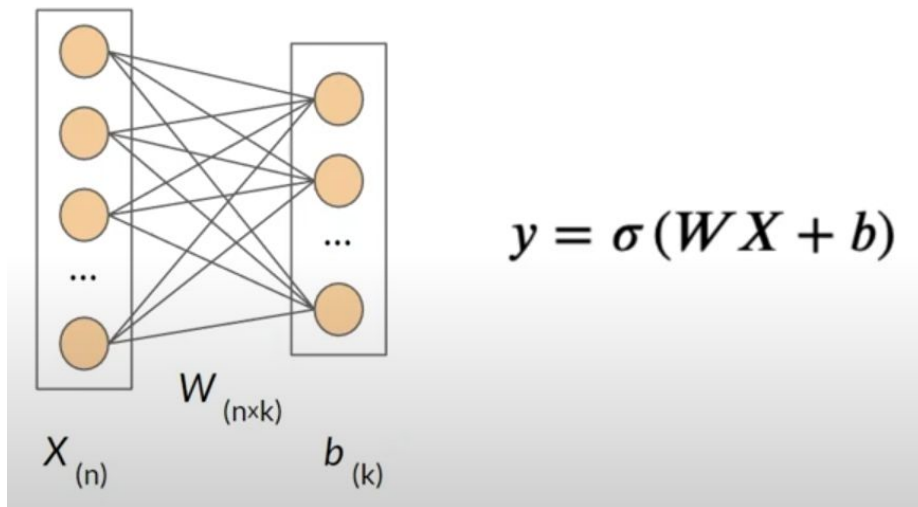
Рекуррентные нейросети имеют скрытые состояния, хранящие информацию о предыдущих входах.

На картинке имеется некоторая последовательность X , проходящая через рекуррентную нейросеть. S - значение скрытого состояния на момент t , U - веса скрытого слоя, V - веса выходов, и W - веса скрытых состояний.



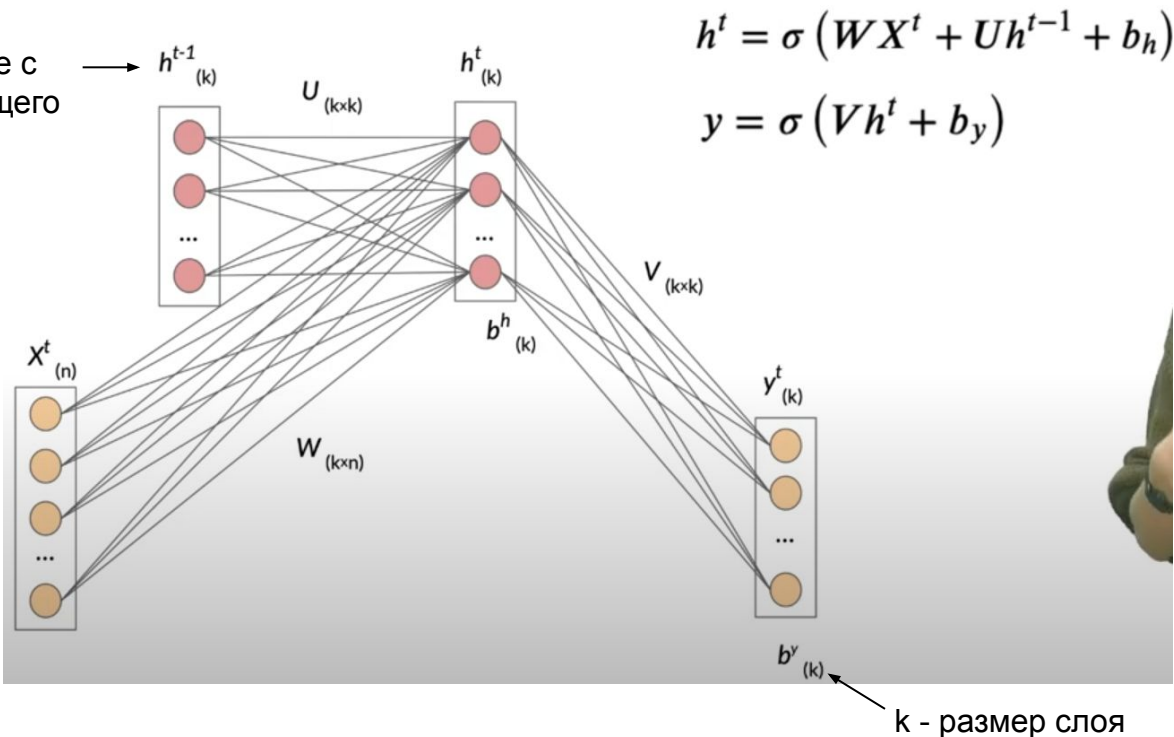
Сравнение с полносвязной нейросетью

Слой полносвязной нейросети



Сравнение с полносвязной нейросетью

Скрытое состояние с предыдущего шага

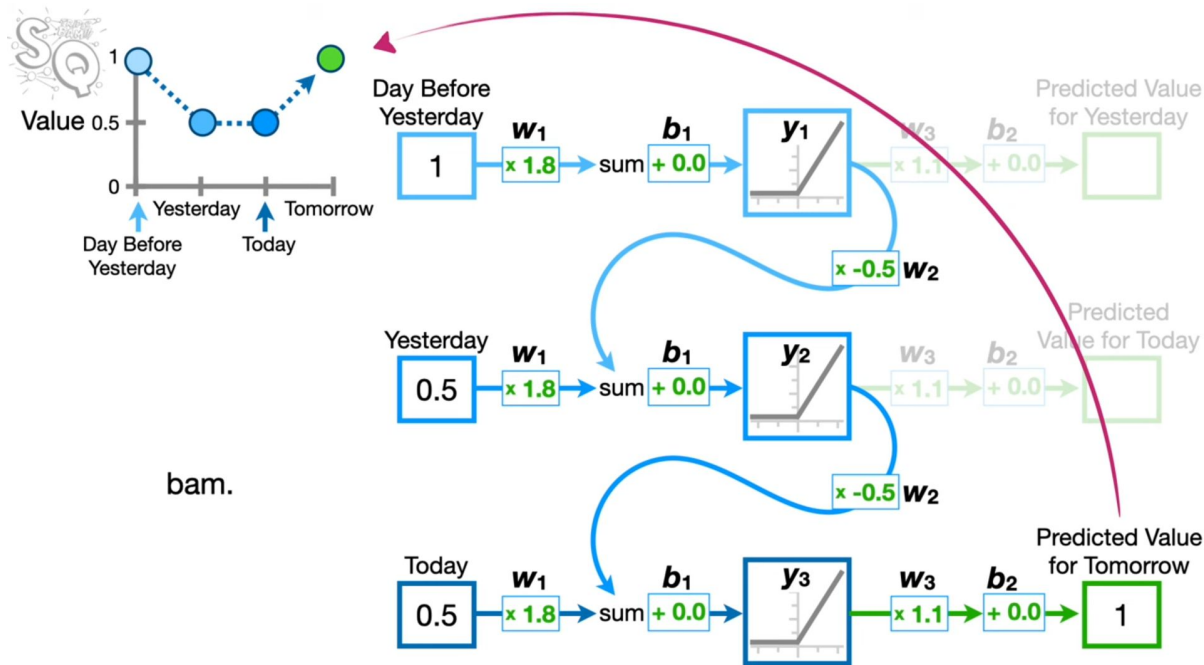


h^t - вектор скрытого состояния текущего токена
 h^{t-1} - вектор скрытого состояния предыдущего токена

Рекуррентные нейросети: архитектура

На картинке справа можно видеть подробный разбор решения задачи предсказания стоимости акций завтра с учетом трех предыдущих дней.

Обратите внимание, что веса и байесы во все дни одинаковы.



Рекуррентные нейросети: взрывающийся/затухающий градиент

В предыдущем примере входящее значение дважды умножается на w_2 . Если этот вес имеет достаточно большое значение, выход (предсказанное значение) данного примера также будет большим. Например, при $w_2 = 4$ нам придется умножать входящее значение на 16. Это увеличит значение градиента. Слишком большое значение градиента означает, что мы проскочим минимум функции потерь, не дойдя до него.

Рекуррентные нейросети: взрывающийся/затухающий градиент

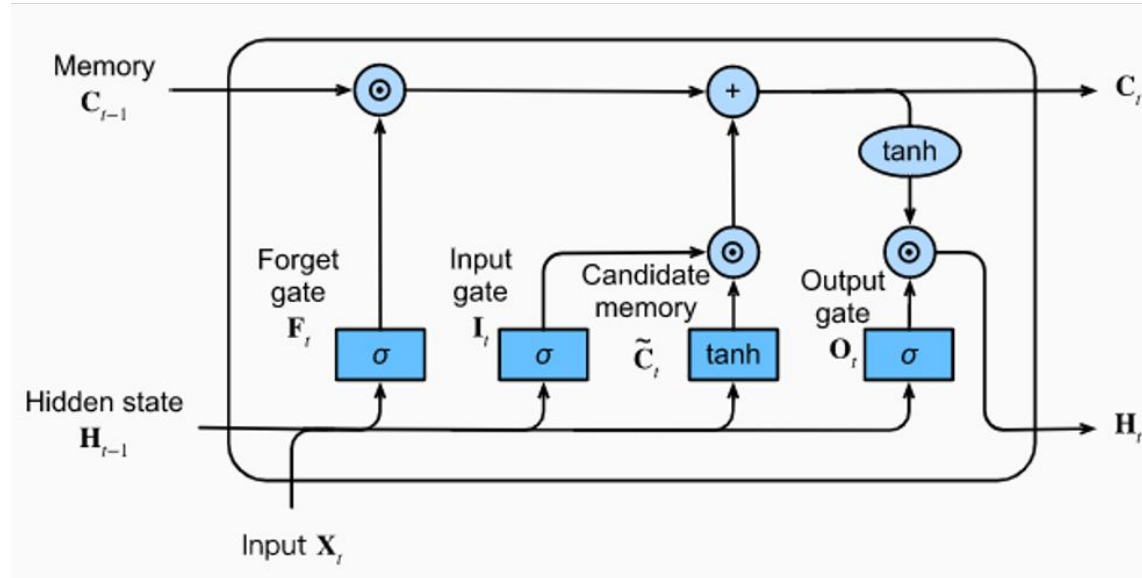
Решением проблемы взрывающегося градиента может быть принудительное занижение весов (gradient clipping). Однако слишком маленькие градиенты тоже являются проблемой: умножение малых чисел дает еще меньшие числа, поэтому шаги градиента становятся слишком маленькими.

Для окончательного решения этой проблемы архитектура рекуррентных нейронных сетей была усовершенствована. Так появились сети с долговременной и кратковременной памятью (LSTM).

LSTM

LSTM (long short term memory networks)

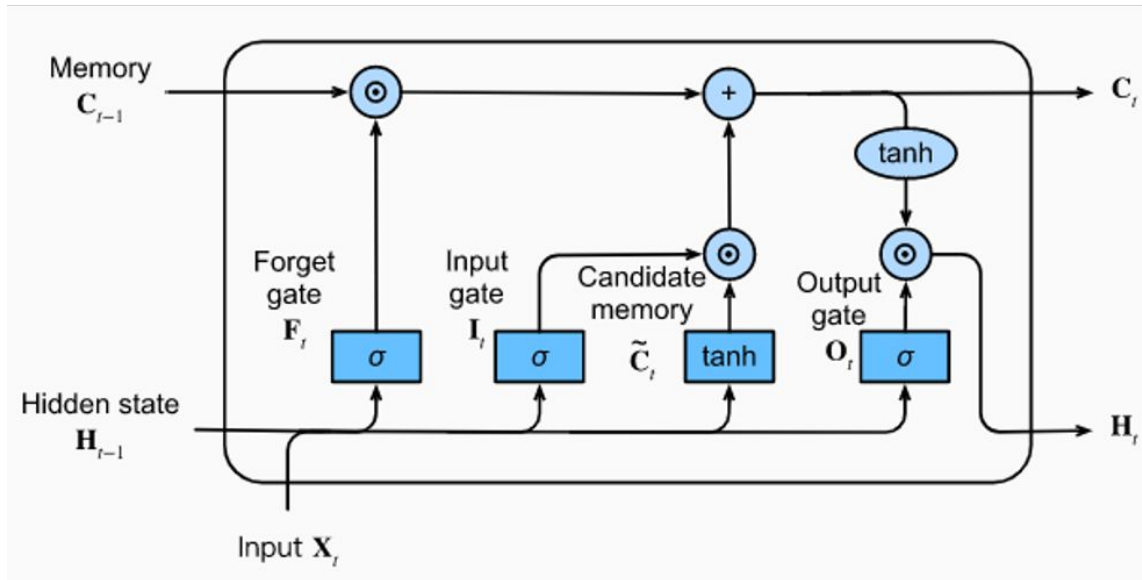
На схеме справа представлен один LSTM-блок. Memory (C) обозначает “долговременную” память, а H - “кратковременную”. Информацию в памяти можно добавлять или удалять при помощи ворот/вентилей (gates).



LSTM (long short term memory networks)

В качестве функций активации в LSTM используются сигмоиды и гиперболические тангенсы.

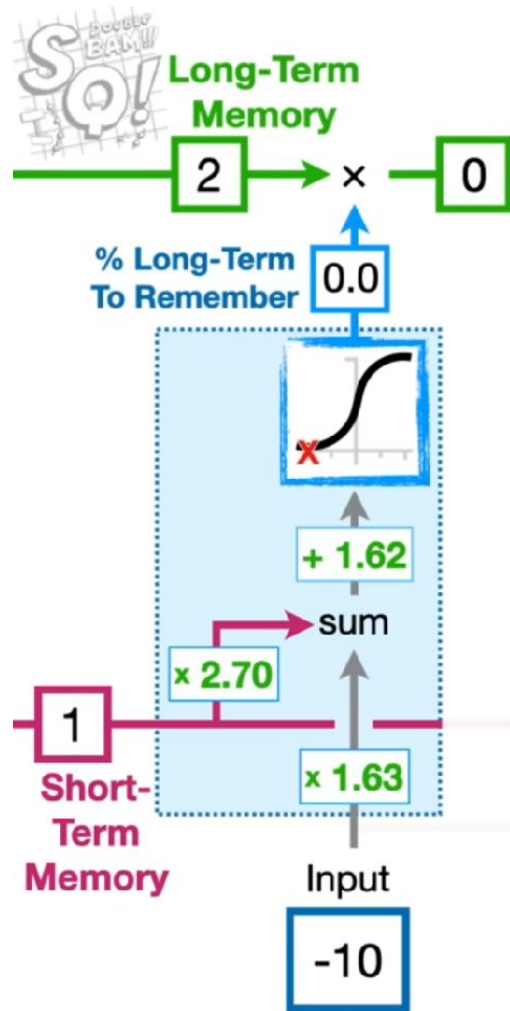
Веса и байесы внутри блока будут одинаковыми для всех блоков.



Forget gate

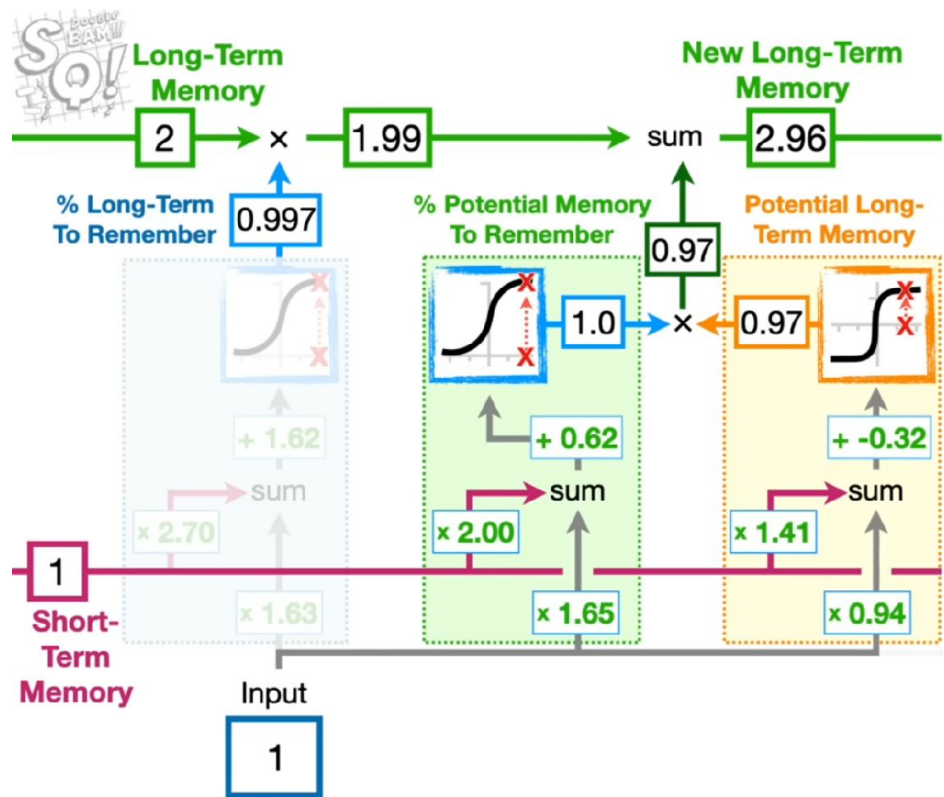
Ворота забывания позволяют на основе предыдущей долговременной и кратковременной памяти, а также входа текущего блока, определить, сколько информации о предыдущих состояниях нужно сохранить.

Так, если текущий вход имеет большое отрицательное значение, сигмоидная функция активации будет равна нулю, вся информация о состоянии предыдущего блока будет забыта.



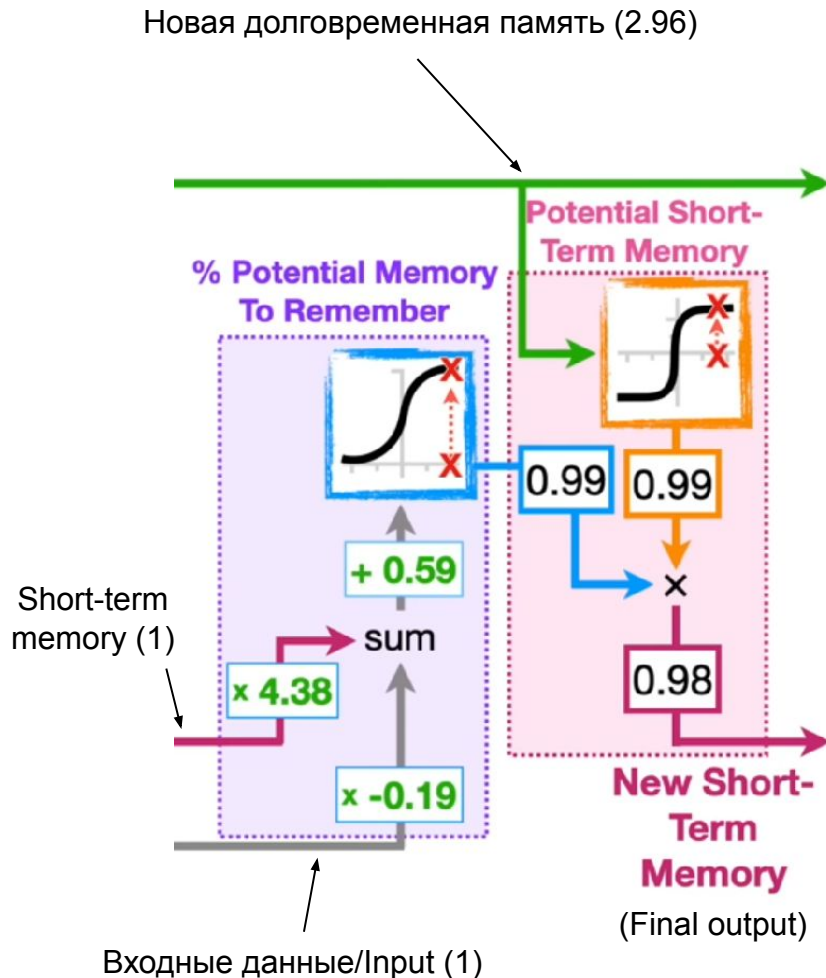
Input gate

Ворота входного состояния состоят из двух частей. Одна, с гиперболическим тангенсом, определяет потенциальную долговременную память. Вторая, идентичная воротам забывания, определяет, сколько потенциальной долговременной памяти действительно нужно запомнить. Выходы этих частей перемножаются, и произведение добавляется к существующей долговременной памяти.



Output gate

Ворота выходного состояния определяют, сколько информации из текущего блока нужно запомнить и передать в следующий. При этом блок, идентичный воротам забывания, на основании входных данных определяет, сколько информации нужно запомнить, а второй блок на основании новой долговременной памяти определяет саму запоминаемую информацию.



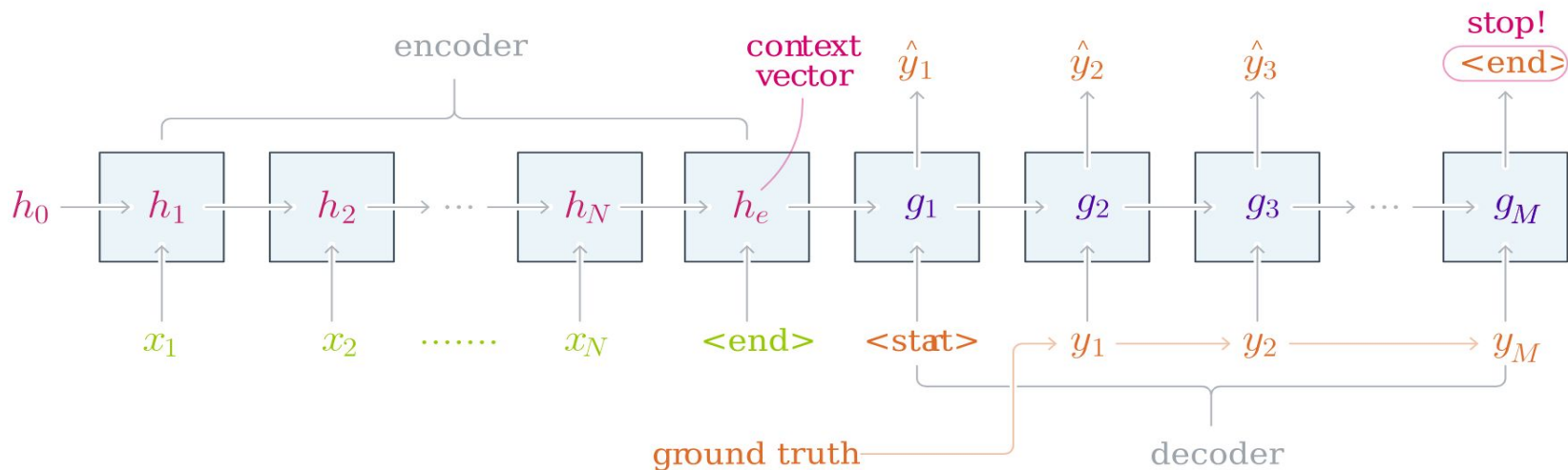
Bidirectional networks

Двунаправленные рекуррентные сети (как RNN, так и LSTM) состоят из двух подсетей: прямой и обратной. В прямую сеть элементы входной последовательности подаются в обычном порядке, в обратную - в обратном. Далее выходы обеих сетей агрегируются (усредняются, суммируются и т.п.).

Двунаправленная сеть может помочь учитывать не только предыдущий контекст, но и последующий.

Архитектура sequence-to-sequence (seq2seq)

Seq2seq-нейросети состоят из двух частей: энкодера (кодировщика) и декодера (декодировщика). Для обоих этих компонентов подходят рекуррентные нейросети.



Энкодер

“Энкодер читает входное предложение token за токеном и обрабатывает их с помощью блоков рекуррентной сети. Hidden state последнего блока становится **контекстным вектором**. Часто энкодер читает предложение в обратном порядке. Это делается для того, чтобы последний token, который видит энкодер, совпал (или примерно совпал) с первыми токенами, которые будет генерировать декодер. Таким образом, декодеру проще начать процесс воссоздания предложения.” (А. Янина, Яндекс.Учебник)

Декодер

Архитектура декодера такая же, как и у энкодера, но вектор скрытого состояния инициализируется при помощи контекстного вектора энкодера. Таким образом, декодер учитывает и сгенерированные им самим токены, и предложение на исходном языке. Первый блок декодера получает на вход метку начала последовательности, все последующие блоки - или сгенерированные декодером слова, или (при тренировке) слова из “истинного” предложения-таргета.



Bam.

Encoder

Layer 2

Layer 1

1 0 0 0

let's to go <EOS>

0 0 1 0

let's to go <EOS>

Decoder

ir vamos y <EOS>

0 1 0 0

SoftMax

-0.9 6.0 -2 -2

1.0 -1.

ir vamos y <EOS>

0 0 0 1

SoftMax

-2 0.2 -2 6.5

0.9 0.8

Практика:

<https://colab.research.google.com/drive/1cieUPQjEPdml0PoS1zijlYXLzj3zsTQu?usp=sharing>

Источники

- Сверточные нейросети:
 - <https://education.yandex.ru/handbook/ml/article/svyortochnye-nejroseti>
 - <https://youtu.be/HGwBXDKFk9I?si=Hw5A1BO53BuHI7B3>
 - <https://www.kaggle.com/code/akhiljethwa/understanding-convolutional-hyperparameters/notebook>
- Рекуррентные нейросети:
 - <https://education.yandex.ru/handbook/ml/article/nejroseti-dlya-raboty-s-posledovatelnostyami>
 - <https://youtu.be/AsNTP8Kwu80?si=RF4c1UFktw9MvH0I>
 - https://youtu.be/YCzL96nL7j0?si=lyRq_5wn-bwoUcwc
 - https://youtu.be/L8HKweZIOmg?si=Yhm9Qbc_rsWyZlcT
 - <https://github.com/bentrevett/pytorch-seq2seq/tree/rewrite>